# Iron Stats: Continuous Biomechanical Tension Modeling, Physiological Fatigue Dynamics, and Multi-Objective Mesocycle Optimization

Lorenzo Zambardi
https://github.com/Lorenzo1901/Iron-Stats

Technical Whitepaper — November 2026

**Abstract**

Conventional resistance-training analytics quantify training stress via coarse discrete scalars—raw set counts, repetition totals, and mechanical tonnage ($L \times R$). Such aggregates implicitly assume uniform mechanical tension across the entire Range of Motion (ROM), identical motor-unit recruitment regardless of proximity to muscular failure, and linear fatigue accumulation across repetitions. **Iron Stats** replaces these scalar heuristics with a unified computational model spanning three layers: (1) *continuous biomechanical tension fields*, parameterizing per-muscle resistance profiles over normalized ROM $x \in [0, 1]$ via area-normalized, strictly monotone cubic Bézier curves; (2) *physiological load and fatigue dynamics*, modeling High-Threshold Motor Unit (HTMU) recruitment via an RPE-gated effective-repetition threshold, piecewise RIR-dependent concentric Time Under Tension (TUT) slowdown, and continuous exponential central nervous system (CNS) fatigue; and (3) *constrained combinatorial optimization*, synthesizing balanced multi-day mesocycles via Monte Carlo scale calibration and scalarized Simulated Annealing. We present a numerical analysis comparing Lagrange interpolation, $C^2$ natural cubic splines, and Bernstein-basis cubic Bézier curves on empirical exercise profiles—explicitly separating boundary jump discontinuities from smooth interior interpolation errors—and prove why Bézier convex-hull containment and $\mathcal{O}(1)$ De Casteljau evaluation eliminate non-physical negative tension while supporting $\mathcal{O}(h^2)$ trapezoidal magnitude conservation.

## 1 Introduction

Quantitative hypertrophy programming traditionally reduces a resistance-training session to discrete scalar summaries:

$$V_{\text{naive}} = \sum_s R_s, \qquad T_{\text{naive}} = \sum_s L_s R_s, \tag{1}$$

where $R_s$ and $L_s$ denote repetitions and external load in set $s$. Despite widespread adoption, these scalars exhibit three structural deficiencies:

1. **Biomechanical Blindness:** Moment arms and external resistance torque vary continuously across joint angles. Dumbbell Lateral Raises (`LR`), for example, peak in resistance near full abduction ($x \to 0.9$), whereas stretch-biased movements peak near $x \to 0$. Scalar tonnage treats both identically.

2. **Effort Invariance:** Two sets of 10 repetitions—one stopped at Rate of Perceived Exertion $\text{RPE} = 5$ (5 Reps in Reserve, $\text{RIR} = 5$) and one pushed to concentric failure ($\text{RPE} = 10$)—yield identical $V_{\text{naive}}$, despite markedly different HTMU recruitment and velocity loss.

3. **Linear Fatigue Assumption:** Systemic CNS and local tissue fatigue grow nonlinearly as repetitions approach failure, accompanied by progressive concentric velocity decay.

To address these gaps, **Iron Stats** models every exercise over a normalized continuous ROM domain $x \in [0, 1]$, integrates per-repetition physiological slowdown and exponential fatigue, and formulates mesocycle synthesis as a constrained multi-objective combinatorial optimization problem. All equations reported herein match the reference implementation in `python_research/analyzer/` (`constants.py`, `models.py`, `metrics.py`, `parser.py`, `solver.py`).

## 2 Biomechanical Exercise Representation

In `models.py`, each exercise $e$ is parameterized by a biomechanical tuple

$$\theta_e = \big(c_{\text{load}},\, f_{\text{base}},\, \mu_{\text{load}},\, q_{\text{load}},\, \text{is_isolation}\big), \tag{2}$$

where $c_{\text{load}} \in (0, 1]$ denotes articular load efficiency, $f_{\text{base}} \in [1, 10]$ is the intrinsic systemic fatigue coefficient, $(\mu_{\text{load}}, q_{\text{load}}) \in \mathbb{R}^+ \times \mathbb{R}$ maps nominal logged load $L_{\text{raw}}$ to effective biomechanical load via

$$L = L_{\text{raw}} \cdot \mu_{\text{load}} + q_{\text{load}}, \tag{3}$$

and $\text{is_isolation} \in \{\text{False}, \text{True}\}$ flags single-joint movements. (For instance, `LR` specifies $\mu_{\text{load}} = 2.0$ to convert per-dumbbell weight into bilateral load.)

Each exercise targets an anatomical sub-muscle set $\mathcal{M}_e = \{m_1, \ldots, m_k\}$ with contribution magnitudes

$\mathbf{M}_e = [M_{e,m_1}, \ldots, M_{e,m_k}]^\top$. To conserve total mechanical work attribution across muscle compartments, `Exercise.__post_init__` enforces the partition-of-unity constraint

$$\sum_{m \in \mathcal{M}_e} M_{e,m} = 1.0, \tag{4}$$

automatically re-normalizing $M_{e,m} \leftarrow \mathrm{round}(M_{e,m}/\sum_j M_{e,j}, 4)$ whenever $|\sum_m M_{e,m} - 1.0| > 10^{-4}$ (accounting for IEEE-754 floating-point tolerances across the 61-exercise JSON catalog).

# 3 Numerical Analysis of Tension Profiles

Representing muscle tension $y(x)$ across normalized ROM $x \in [0,1]$ requires a functional basis that is parsimonious for interactive editing, smooth, numerically stable, and physically non-negative ($y(x) \geq 0$, as skeletal muscle cannot generate repulsive active tension). We evaluate three candidate numerical frameworks on the `LR` (*Lateral Deltoid*) reference profile defined in `defaultExercises.json` ($P_0 = (0.20, 0.00)$, $P_1 = (0.33, 0.30)$, $P_2 = (0.66, 0.80)$, $P_3 = (0.90, 1.00)$, $M = 0.90$).

The reference `LR` profile is $y(x) = 0$ for $x < x_0$ and $x > x_3$, with $y(x_3^-) = 1.0$ and $y(x_3^+) = 0.0$ (and analogously a slope discontinuity at $x_0 = 0.20$). Any continuous global interpolant (Lagrange or $C^2$ cubic spline) on $[0,1]$ must therefore exhibit a Gibbs-like overshoot near $x_3$—and undershoot near $x_0$—independent of node spacing. The large oscillations visible in full-domain plots across $x \in [0.8, 1.0]$ are *Gibbs oscillations driven by the discontinuity*, not Runge oscillations driven by node clustering. To isolate pure approximation error from the discontinuity, each comparison below pairs the full domain $[0,1]$ with a restricted smooth interior domain $[x_0+\delta, x_3-\delta] = [0.25, 0.85]$ ($\delta = 0.05$).

## 3.1 Lagrange Interpolation and Runge vs. Gibbs Effects

Given $n$ sampled ROM-tension nodes $\{(x_i, y_i)\}_{i=1}^n$, the unique degree-$(n-1)$ global interpolating polynomial is

$$p_{n-1}(x) = \sum_{i=1}^{n} y_i \ell_i(x), \qquad \ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}. \tag{5}$$

On uniform nodes, the Vandermonde system $V\mathbf{a} = \mathbf{y}$ reaches condition number $\kappa(V) = 2.01 \times 10^6$ at $n = 9$, and the Lebesgue constant grows exponentially as $\Lambda_n \sim \mathcal{O}(2^n)$. On the full domain $[0,1]$, equispaced Lagrange interpolation ($n = 9$) yields $L^\infty$ error $0.960$, dominated by the jump at $x_3 = 0.90$ and resting-zone ripples in $[0, 0.20]$. On the restricted smooth interior $[0.25, 0.85]$, however, the $L^\infty$ error drops to $2.90 \times 10^{-5}$ (Fig. 1).

Replacing equispaced nodes with Chebyshev–Gauss–Lobatto (CGL) nodes mapped to $[a,b]$,

$$x_k = a + \frac{b-a}{2}\left(1 - \cos\frac{k\pi}{N-1}\right), \qquad k = 0, \ldots, N-1, \tag{6}$$

reduces Lebesgue growth to $\Lambda_N \sim \frac{2}{\pi}\ln N + \mathcal{O}(1)$. On the smooth interior $[0.25, 0.85]$, CGL achieves $L^\infty = 2.07 \times 10^{-4}$ ($N = 6$) and $5.42 \times 10^{-7}$ ($N = 12$). Yet on the full physical domain $[0,1]$, global coupling still yields $L^\infty = 0.988$ ($N = 6$) and $0.804$ ($N = 12$) with negative dips ($y(x) < 0$) in the resting zone $x \in [0, 0.2]$ (Fig. 2), making polynomial interpolation unsuitable for bounded-support exercise profiles.

## 3.2 $C^2$ Natural Cubic Splines

Piecewise natural cubic splines partition $[a,b]$ using $N$ knots into $N-1$ subintervals $[x_k, x_{k+1}]$, enforcing $C^2$ interior continuity and zero boundary curvature $S''(x_0) = S''(x_{N-1}) = 0$. Solving the tridiagonal system $A\mathbf{m} = \mathbf{d}$ via the Thomas algorithm takes $\mathcal{O}(N)$ operations and minimizes $\int_a^b [S''(x)]^2\,dx$.

On the restricted smooth interior $[0.25, 0.85]$, a 6-knot natural $C^2$ spline attains $L^\infty = 2.46 \times 10^{-3}$ without oscillation. On the full domain $[0,1]$, however, $C^2$ stiffness produces non-physical negative undershoot ($y(x) < 0$) across the flat-to-ramp transition $x \in (0, 0.2)$ alongside Gibbs overshoot ($L^\infty = 0.995$) at $x_3$ (Fig. 3). Suppressing undershoot requires either dense knots ($N \geq 10$) or relaxing continuity to $C^1$ monotone piecewise cubic Hermite interpolation (PCHIP), sacrificing parameter parsimony or $C^2$ curvature (Fig. 4).

## 3.3 Cubic Bézier Curves, Monotonicity, and Bernstein Guarantees

In `BezierProfile` (`models.py`), each active tension profile over $[x_0, x_3] \subseteq [0,1]$ is defined parametrically by four absolute 2D control points $\mathbf{P}_i = (x_i, y_i)^\top \in [0,1]^2$, $i \in \{0,1,2,3\}$, in the cubic Bernstein basis (and $y(x) = 0$ for $x \notin [x_0, x_3]$):

$$\mathbf{P}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix} = \sum_{i=0}^{3} \mathbf{P}_i B_{i,3}(t), \qquad t \in [0,1], \tag{7}$$

where $B_{i,3}(t) = \binom{3}{i} t^i (1-t)^{3-i}$. Explicitly, both components share the same convex combination:

$$x(t) = (1-t)^3 x_0 + 3(1-t)^2 t\, x_1 + 3(1-t)t^2\, x_2 + t^3 x_3, \tag{8}$$

$$y(t) = (1-t)^3 y_0 + 3(1-t)^2 t\, y_1 + 3(1-t)t^2\, y_2 + t^3 y_3. \tag{9}$$

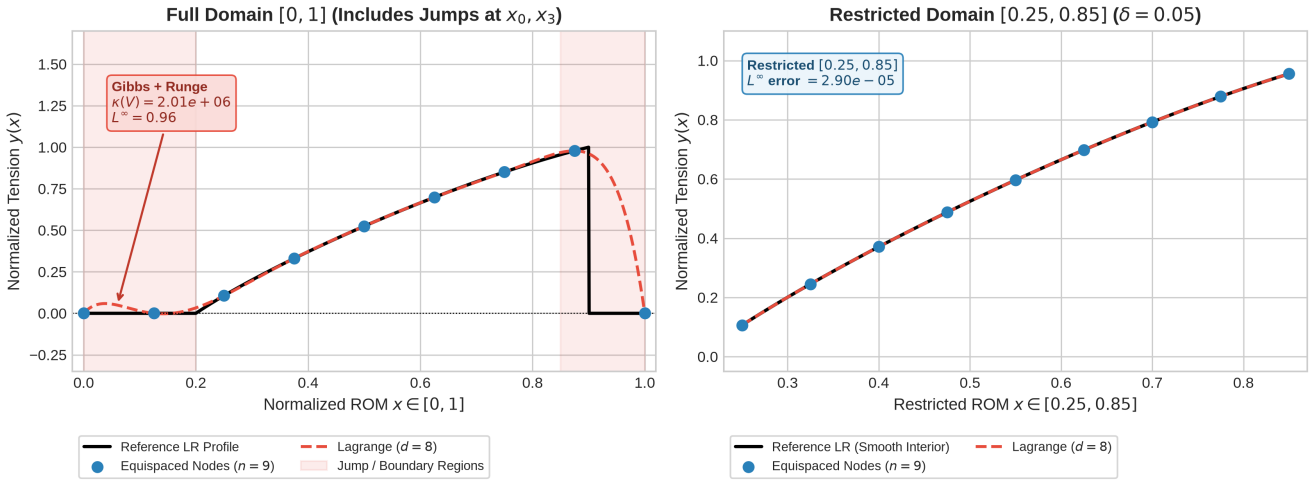



Figure 1: Lagrange interpolation ($n = 9$ equispaced nodes) on LR. Left: full domain $[0, 1]$, showing Gibbs overshoot at $x_3 = 0.90$ and resting-zone oscillations ($L^\infty = 0.960$, $\kappa(V) = 2.01 \times 10^6$). Right: restricted domain $[0.25, 0.85]$ ($\delta = 0.05$), isolating smooth interior error ($L^\infty = 2.90 \times 10^{-5}$).

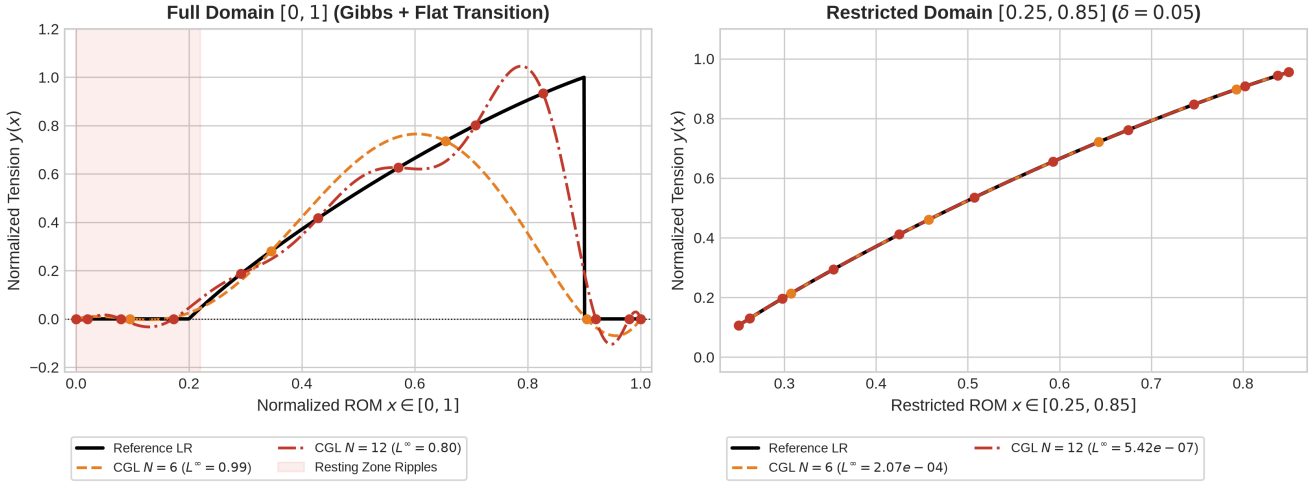



Figure 2: Chebyshev–Gauss–Lobatto interpolation ($N = 6, 12$) on LR. Left: full domain $[0, 1]$, showing Gibbs error at $x_3$ and negative ripples in $x \in [0, 0.2]$ ($L^\infty = 0.988$ for $N = 6$, $0.804$ for $N = 12$). Right: restricted domain $[0.25, 0.85]$ ($L^\infty = 2.07 \times 10^{-4}$ and $5.42 \times 10^{-7}$).

**Theorem 1** (Convex Hull Non-Negativity). *For all $t \in [0, 1]$, the cubic Bernstein polynomials satisfy $B_{i,3}(t) \geq 0$ and form a partition of unity $\sum_{i=0}^{3} B_{i,3}(t) = \big((1-t)+t\big)^3 \equiv 1$. Consequently, $\mathbf{P}(t) \in \mathrm{Conv}(\{\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3\})$, and if $y_i \geq 0$ for all $i \in \{0, 1, 2, 3\}$, then $y(t) \geq 0$ for every $t \in [0, 1]$.*

*Proof.* Since $t \geq 0$ and $1 - t \geq 0$ on $[0, 1]$, each $B_{i,3}(t) \geq 0$. By the binomial theorem, $\sum_{i=0}^{3} \binom{3}{i} t^i (1-t)^{3-i} = 1$, so $y(t) = \sum_{i=0}^{3} B_{i,3}(t) y_i$ is a convex combination of $\{y_i\}$ and satisfies $\min_i y_i \leq y(t) \leq \max_i y_i$. Setting $y_i \geq 0$ guarantees $y(t) \geq 0$ identically. □

Furthermore, $\mathbf{P}(t)$ is evaluated stably in $\mathcal{O}(1)$ time without division via De Casteljau's recursive convex linear interpolations (Fig. 5):

$$\begin{aligned} \mathbf{Q}_i(t) &= (1-t)\mathbf{P}_i + t\mathbf{P}_{i+1}, && i \in \{0, 1, 2\}, \\ \mathbf{R}_i(t) &= (1-t)\mathbf{Q}_i(t) + t\mathbf{Q}_{i+1}(t), && i \in \{0, 1\}, \\ \mathbf{P}(t) &= (1-t)\mathbf{R}_0(t) + t\mathbf{R}_1(t), \end{aligned}$$

where $\mathbf{R}_0(t)\mathbf{R}_1(t)$ gives the exact tangent direction and endpoints obey $\mathbf{P}'(0) = 3(\mathbf{P}_1 - \mathbf{P}_0)$ and $\mathbf{P}'(1) = 3(\mathbf{P}_3 - \mathbf{P}_2)$.

While `BezierProfile.get_curve` evaluates (8)–(9) directly over $t \in [0, 1]$, querying tension at a physical ROM coordinate $x_{\text{target}} \in [x_0, x_3]$ requires inverting $x(t^*) = x_{\text{target}}$ (implemented in `models.py::BezierProfile.solve_y` and `helpers.js::solveBezierY`). The inversion $x(t^*) = x_{\text{target}}$ is well-posed only if $x_0 < x_1 < x_2 < x_3$; under this condition $x'(t) > 0$ on $(0, 1)$, so $x(t)$ is strictly increasing on

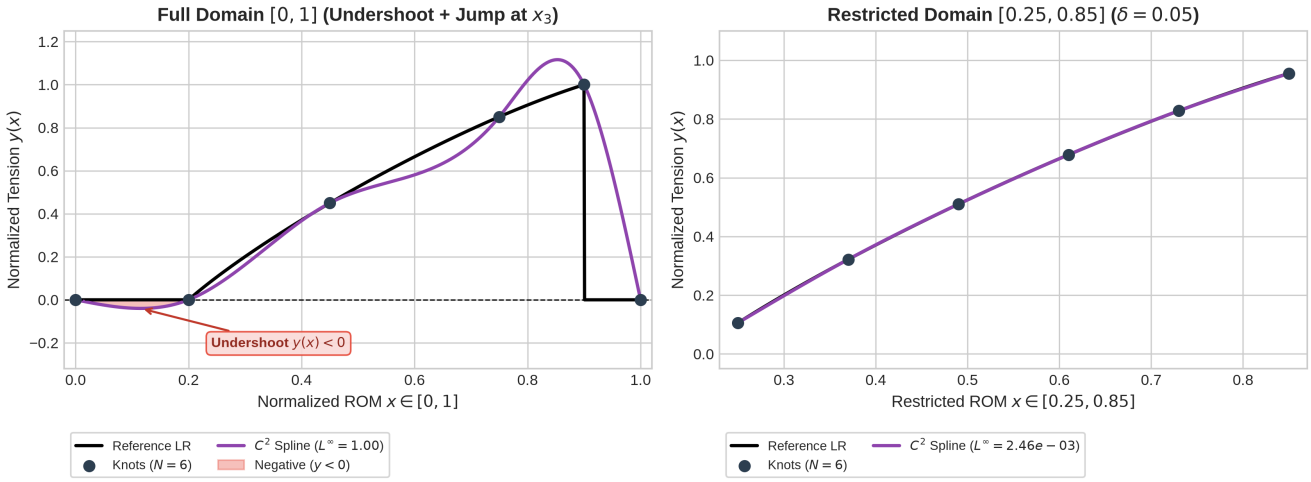



Figure 3: Natural $C^2$ cubic spline ($N = 6$ knots) on LR. Left: full domain $[0, 1]$, exhibiting non-physical negative undershoot ($y(x) < 0$) in $(0, 0.2)$ and Gibbs overshoot at $x_3$ ($L^\infty = 0.995$). Right: restricted domain $[0.25, 0.85]$ ($L^\infty = 2.46 \times 10^{-3}$).

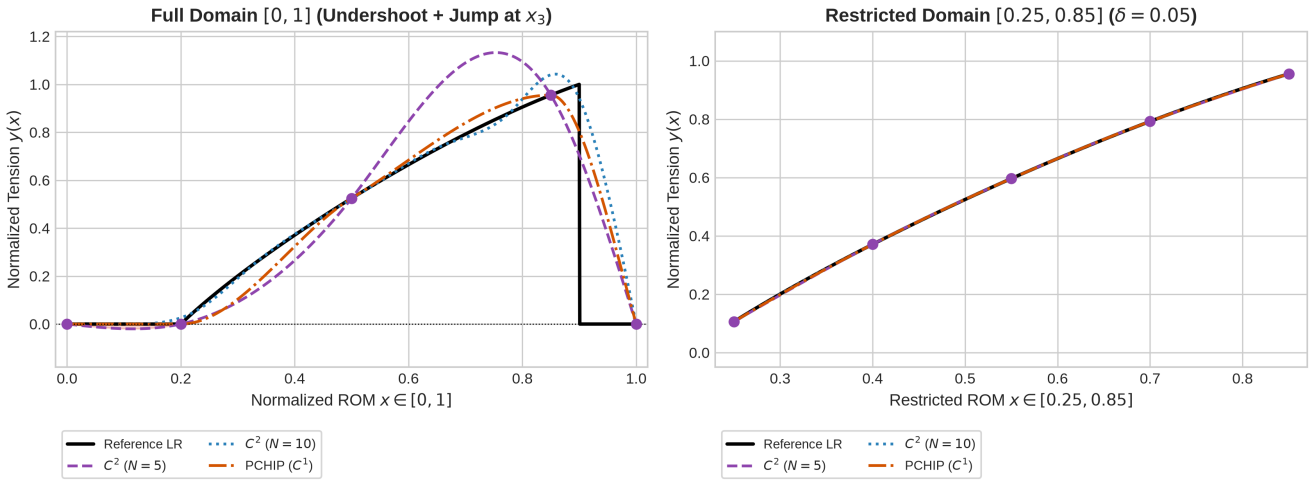



Figure 4: Spline trade-off on LR. Left: full domain $[0, 1]$ ($N = 5$ undershoots, $N = 10$ inflates user knots, PCHIP reduces to $C^1$). Right: restricted domain $[0.25, 0.85]$ showing smooth interior agreement.

$[0, 1]$ and $t^*$ is unique. This precondition is enforced at load and edit time in `BezierProfile.__post_init__`, `WorkoutSolver` (`solver.js`), `solveBezierY`, and `BezierEditor.jsx`, and across all 61 exercises in `defaultExercises.json`. After $K$ bisection iterations on $t \in [0, 1]$,

$$|t^{(K)} - t^*| \le 2^{-K}, \quad |x(t^{(K)}) - x_{\text{target}}| \le 2^{-K} \max_{[0,1]} |x'(t)|, \tag{10}$$

so bounding $t$ controls $x$-error up to the local slope $|x'(t)|$; both `solve_y` and `solveBezierY` run up to $K = 30$ iterations with early exit at $|x(t^{(k)}) - x_{\text{target}}| < 10^{-6}$. Fig. 6, Fig. 7, and Table 1 summarize the Bernstein basis and method comparison.

Table 1: Numerical properties of ROM tension approximations.

| Method | Params | Cost | $y(x) \ge 0$ | Support $[x_0, x_3]$ |
|---|---|---|---|---|
| Lagrange (Equi.) | $n$ nodes | $\mathcal{O}(n^2)$ | No | Gibbs at jump |
| Lagrange (CGL) | $N \ge 12$ | $\mathcal{O}(n \log n)$ | No | Resting ripples |
| Natural $C^2$ Spline | $N$ knots | $\mathcal{O}(N)$ | No | Undershoot $x_0$ |
| **Cubic Bézier** | **4 pts** | $\boldsymbol{\mathcal{O}(1)}$ | **Yes** | **Exact** $[x_0, x_3]$ |

## 3.4 Non-Uniform Composite Trapezoidal Quadrature

To conserve muscle contribution magnitude $M_{e,m}$ under arbitrary control-point shapes, `BezierProfile.get_curve` discretizes $t_k = k/(R-1)$ ($R = 50$, $h = 1/(R-1)$) and integrates $y\,dx$ with $\Delta x_k = x(t_{k+1}) - x(t_k)$ via the composite trapezoidal rule:

$$I_{\text{trap}} = \sum_{k=0}^{R-2} \frac{y(t_k) + y(t_{k+1})}{2} \Delta x_k. \tag{11}$$

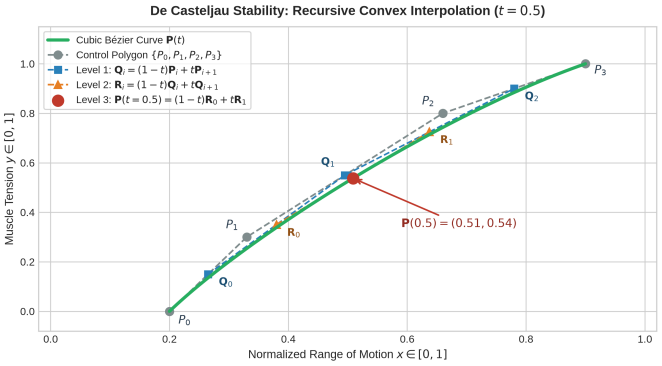



Figure 5: De Casteljau geometric construction at $t = 0.5$ on `LR`, requiring six division-free convex linear interpolations in $\mathcal{O}(1)$ time.

Because $t_k$ is uniform while $x(t_k)$ is *non-uniformly spaced*, the classical uniform-grid formula $-\frac{(x_3-x_0)^3}{12(R-1)^2}y''(\xi)$ does *not* apply. Under strict monotonicity ($x_0 < x_1 < x_2 < x_3$, ensuring $\Delta x_k > 0$), the truncation error over non-uniform nodes is

$$E_{\text{trap}} = -\frac{1}{12}\sum_{k=0}^{R-2} y''(\xi_k)\,(\Delta x_k)^3, \tag{12}$$

with $\xi_k \in [x(t_k), x(t_{k+1})]$, or equivalently in terms of the parametric integrand $g(t) = y(t)x'(t)$ with uniform step $h = 1/(R-1)$,

$$E_{\text{trap}} = -\frac{h^2}{12}\int_0^1 g''(t)\,dt + \mathcal{O}(h^4). \tag{13}$$

The curve is then rescaled as $B_{e,m}(x) = y(t^*(x)) \cdot \frac{M_{e,m}}{I_{\text{trap}}}$ for $x \in [x_0, x_3]$ (or 0 if $I_{\text{trap}} \le 0$ or $x \notin [x_0, x_3]$), guaranteeing discrete area conservation on the parametric quadrature grid and continuous convergence to $M_{e,m} + \mathcal{O}(h^2)$.

# 4 Physiological Dynamics & Optimization

## 4.1 Effective Volume, Dynamic TUT, and CNS Fatigue

In `constants.py` and `models.py` (`SetData`), each set contains $n_b$ base repetitions, $n_a$ assisted repetitions ($\alpha_{\text{ass}} = 0.5$), and $n_p$ partial repetitions ($\alpha_{\text{part}} = 0.33$). Both `parser.py` and `metrics.py` accept any RPE $\ge 0$ (defaulting omitted set RPE to 9.0); by contrast, the simulated-annealing solver in `solver.py` clamps candidate RPE values to $[5.0, 10.0]$ during mutation as a search-space restriction, not a data constraint. Total and HTMU-effective repetitions are:

$$n_{b,\text{eff}} = \min\big(n_b, \max(0, \text{RPE} - 4.0)\big), \tag{14}$$

$$R_{\text{tot}} = n_b + \alpha_{\text{ass}} n_a + \alpha_{\text{part}} n_p, \tag{15}$$

$$R_{\text{eff}} = n_{b,\text{eff}} + \alpha_{\text{ass}} n_a + \alpha_{\text{part}} n_p. \tag{16}$$

Base repetitions recruit high-threshold motor units when RIR $< 6$ (RPE $> 4.0$).

In `metrics.py` (`calculate_set_tuts`), given execution tempo $(c, p_s, e, p_l)$ (default 1-0-2-0 s) with nominal duration $\tau_0 = c + p_s + e + p_l$, each base repetition $j \in \{0, \dots, n_b - 1\}$ experiences instantaneous $\text{RIR}_j = (10.0 - \text{RPE}) + (n_b - 1 - j)$ and concentric slowdown $\Delta c(\text{RIR}_j)$:

$$\Delta c(\text{RIR}_j) = \begin{cases} 1.60\,\text{s}, & \text{RIR}_j < 1.0, \\ 1.00\,\text{s}, & 1.0 \le \text{RIR}_j < 2.0, \\ 0.60\,\text{s}, & 2.0 \le \text{RIR}_j < 3.0, \\ 0.35\,\text{s}, & 3.0 \le \text{RIR}_j < 4.0, \\ 0.15\,\text{s}, & 4.0 \le \text{RIR}_j < 5.0, \\ 0.00\,\text{s}, & \text{RIR}_j \ge 5.0, \end{cases} \tag{17}$$

yielding per-rep TUT $\tau_j = \tau_0 + \Delta c(\text{RIR}_j)$ for base reps, $\tau_0$ for assisted reps, and $0.5\tau_0$ for partial reps. Set `total_tut` and `effective_tut` are computed during logbook ingestion in `parser.py::analyze_workout_log`, where `effective_tut` sums over the last $\lfloor n_{b,\text{eff}} + 0.5 \rfloor$ base reps plus all assisted and partial reps.

In `metrics.py` (`calculate_set_fatigue`), per-rep effort $\text{RPE}_j = \max(0, 10.0 - \text{RIR}_j)$ drives a continuous exponential CNS fatigue multiplier $\mu(\text{RPE}_j) = 1.1^{\text{RPE}_j - 7.5}$ for all $\text{RPE}_j \ge 0$:

$$\Phi_{\text{set}} = F_{\text{ex}} K_{\text{load}} \Bigg[ \sum_{j=0}^{n_b-1} \tau_j\, 1.1^{\text{RPE}_j - 7.5} + \alpha_{\text{ass}} 1.1^{-0.5} n_a \tau_0 + \alpha_{\text{part}} n_p \tau_0 \Bigg], \tag{18}$$

where $F_{\text{ex}} = f_{\text{base}}$ and $K_{\text{load}} = c_{\text{load}}$. All set-level volume, tonnage ($LR_{\text{tot}}$), weighted tonnage ($LR_{\text{tot}} c_{\text{load}}$), TUT, and fatigue totals are projected onto queried macro- or sub-muscles via $M_{e,m}$ in `metrics.py::calculate_metrics`.

## 4.2 Constrained Multi-Objective Combinatorial Solver

In `solver.py` (`WorkoutSolver`), a candidate mesocycle $x \in \mathcal{X}$ assigns $D$ training days, each containing $N_d \in [N_{\min}, N_{\max}]$ exercises, $S_{d,e} \in [S_{\min}, S_{\max}]$ sets, base reps $r \in [r_{\min}, r_{\max}]$, RPE $\in \{7.0, 8.0, 8.5, 9.0, 10.0\}$ (mutated within $[5.0, 10.0]$), and stretched-ROM partials $p \in \{0, 2, 4, 6\}$ (enabled when RPE $\ge 9.0$). Let $x_i = i/(R-1)$, $i = 0, \dots, R-1$ ($R = 50$), be a uniform grid on physical ROM $[0, 1]$. For each exercise $e$, let $t_{e,i} = x_e^{-1}(x_i)$ be the inverted parameter when $x_i \in [x_{0,e}, x_{3,e}]$, evaluated via `BezierProfile.get_physical_curve`. Writing $N_{b,e} = \sum_{s \in e} n_{b,e,s}$ and $N_{p,e} = \sum_{s \in e} n_{p,e,s}$ for exercise-level rep sums over sets, the aggregate tension curve for

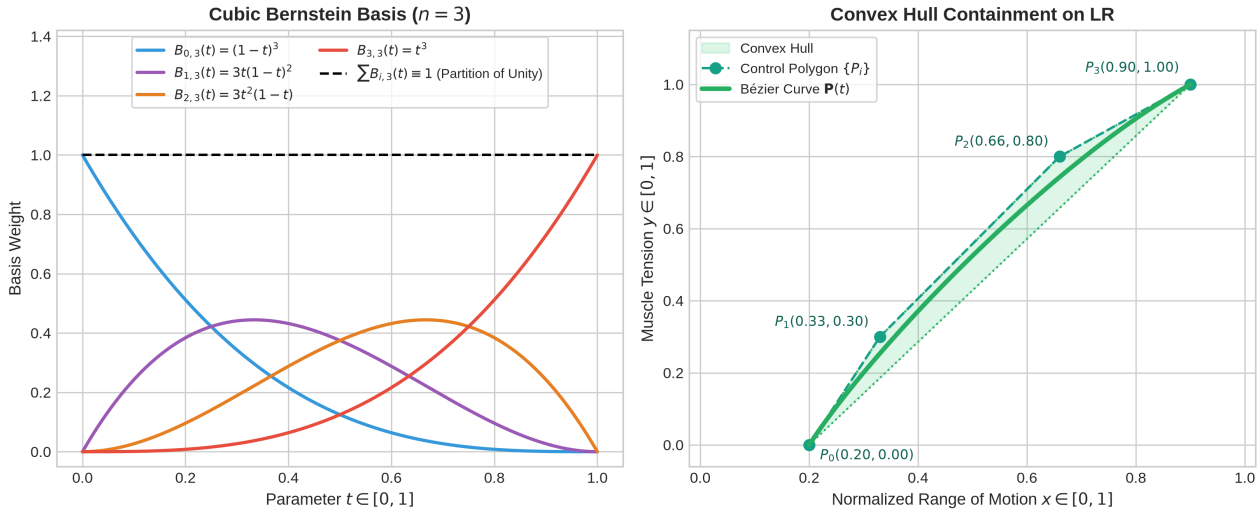



Figure 6: Left: Cubic Bernstein basis polynomials $B_{i,3}(t)$ verifying $\sum_{i=0}^{3} B_{i,3}(t) \equiv 1$. Right: Resulting `LR` curve enclosed strictly inside the convex hull of its four control points.

muscle $m$ is

$$V_m(x_i) = \sum_{d=1}^{D} \sum_{e \in x_d} B_{e,m}(x_i)\big(N_{b,e} + \alpha_{\text{part}} N_{p,e} \chi_{\text{part}}(x_i)\big), \tag{19}$$

where $\chi_{\text{part}}(x_i) = \mathbb{I}[x_i \le 0.3333]$ restricts partial repetitions to the stretched third of physical ROM and $\alpha_{\text{part}} = 0.33$ matches the partial weighting used in muscle volume totals.

**Fast SA Surrogates.** During Simulated Annealing search, `evaluate_workout` replaces the full §4.1 logbook models with three lightweight surrogates chosen for speed (not intended for final reporting):

- *Load:* $\hat{L} = \mu_{\text{load}} \frac{100.0}{1.0 + 0.033\, n_b} + q_{\text{load}}$ (Epley-like 1RM scaling).
- *TUT:* constant $2.0\,\text{s/rep}$, i.e. $\text{TUT}_{\text{tot}} = 2.0 R_{\text{tot}}$ and $\text{TUT}_{\text{eff}} = 2.0 R_{\text{eff}}$. Because the factor $2.0$ cancels in $\text{TUT}_{\text{eff}}/\text{TUT}_{\text{tot}}$, $E_{\text{tut}} \equiv E_{\text{vol}}$ identically during search.
- *Fatigue:* linear $0.01 R_{\text{tot}} \hat{L} f_{\text{base}} c_{\text{load}}$, omitting exponential RPE scaling and dynamic TUT.

Letting $\hat{V}_m(x_i) = V_m(x_i)/\max_r V_m(x_r)$, seven dimensionless error metrics $\mathcal{I} = \{\text{curve}, \text{vol}, \text{ton}, \text{tut}, \text{distr}, \text{variety}, \text{balance}\}$ evaluate mesocycle quality:

$$E_{\text{curve}} = \frac{1}{|\mathcal{M}_{\text{tr}}| R} \sum_{m \in \mathcal{M}_{\text{tr}}} \sum_{i=0}^{R-1} \big(\hat{V}_m(x_i) - T_m(x_i)\big)^2, \tag{20}$$

$$E_q = \left(\frac{X_{q,\text{eff}}}{X_{q,\text{tot}}} - \rho_q^*\right)^2, \qquad q \in \{\text{vol}, \text{ton}, \text{tut}\}, \tag{21}$$

$$E_{\text{distr}} = \frac{1}{2} \sum_m \big(p_m^{\text{actual}} - p_m^{\text{target}}\big)^2, \tag{22}$$

$$E_{\text{variety}} = \left(1 - \frac{|\mathcal{U}(x)|}{N_{\text{tot}}(x)}\right)^2, \tag{23}$$

$$E_{\text{balance}} = \frac{1}{2}\left(\frac{\text{Var}_d(F_d)}{(F_{\max}/2)^2} + \frac{\text{Var}_d(V_d)}{(V_{\max}/2)^2}\right), \tag{24}$$

where $T_m \in \{\text{constant}, \text{sigmoid}, \text{inv_sigmoid}, \text{linear}\}$, default $\rho_q^* = 0.70$, $p_m^{\text{target}}$ is the user-configured target share from `volume_dist`, $p_m^{\text{actual}} = V_m / \sum_{m'} V_{m'}$ is the actual share of effective muscle-weighted total volume, $\text{Var}_d(\cdot)$ denotes the population variance (`np.var`, ddof $= 0$) across days $d = 1, \ldots, D$, $F_{\max} = 15\, N_{\max} S_{\max} r_{\max}$, and $V_{\max} = N_{\max} S_{\max} r_{\max}$.

Prior to optimization, `calibrate()` estimates normalization denominators via the empirical sample mean $\hat{\mu}_i = \frac{1}{K} \sum_{r=1}^{K} E_i(x^{(r)})$ over $K = 500$ random states, setting $s_i = 2\hat{\mu}_i$ (with $\mathcal{O}(K^{-1/2})$ sampling variance and zero-guard $s_i \leftarrow 10^{-6}$ if $s_i = 0$). `solve()` then minimizes

$$\min_{x \in \mathcal{X}} J(x) = 10^4 \sum_{i \in \mathcal{I}} w_i \frac{E_i(x)}{s_i} + 10^3\, P_{\text{dup}}(x), \tag{25}$$

where $P_{\text{dup}}(x) = 150 \cdot N_{\text{dup}}(x)$ penalizes same-day duplicate exercises, using Simulated Annealing with eight discrete mutation operators, unified initial temperature $T_0 = 50.0$, geometric cooling $T_{t+1} = \gamma T_t$ ($\gamma = (0.01/T_0)^{1/N_{\text{iter}}}$), and Metropolis acceptance $\exp(-\Delta J / T_t)$.

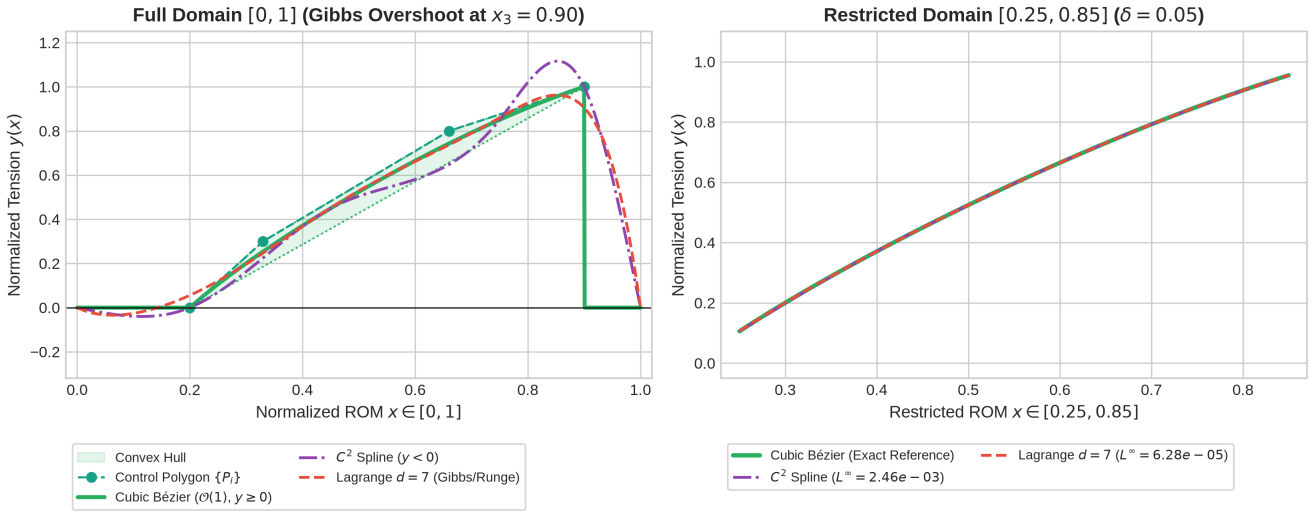



Figure 7: Synoptic comparison on LR. Left: full domain $[0, 1]$, showing Gibbs overshoot at $x_3 = 0.90$ for continuous global interpolants. Right: restricted domain $[0.25, 0.85]$ ($\delta = 0.05$), isolating smooth interior errors ($L^\infty = 6.28 \times 10^{-5}$ for Lagrange $d = 7$, $2.46 \times 10^{-3}$ for $C^2$ spline).

# 5 Conclusions & Software Architecture

Iron Stats replaces scalar training logs with strictly monotone, Bernstein-basis Bézier tension fields, RIR-aware concentric slowdown, continuous exponential CNS fatigue, and Monte Carlo calibrated Simulated Annealing. The Python analyzer (`python_research/analyzer/`, packaged via `pyproject.toml` and `requirements.txt` with `numpy` as sole runtime dependency) and the React/Vite/Capacitor frontend (`frontend/`) share the same Bézier curve definition (7)–(9) and the same solver objective. They differ intentionally in two client-UI implementation details: (i) the analyzer and solver integrate $\int y\,dx$ parametrically over $t$ by the trapezoidal rule, whereas the dashboard chart (`DashboardTab.jsx`) samples $y$ on a 51-point uniform $x$-grid and normalizes by a Riemann sum with $1/50$ spacing (agreeing within $\sim 10^{-2}$ on LR, verified by automated parity tests); and (ii) the interactive curve editor (`BezierEditor.jsx`) normalizes its SVG preview by peak height $y/\max y$ rather than area. Future work will collect longitudinal training datasets to empirically calibrate weights $w_i$ and integrate IMU or computer-vision kinematics to refine exercise-specific $B_{e,m}(x)$ profiles.